

WALMART CASE STUDY

Walmart Black Friday Customer Analytics

✓ 1 Problem Statement + Basic Metrics

1. Problem Statement

Walmart wants to understand customer purchasing behavior during Black Friday. The primary objective is to analyze the purchase amount across different customer segments such as gender, age, marital status, city category, occupation, and product category.

A key business question is:

Do female customers spend more per transaction than male customers on Black Friday?

The analysis will use sample transaction data to compare customer spending patterns and use statistical techniques such as the Central Limit Theorem and Confidence Intervals to make inferences about the larger customer population.

✓ 2. Load & Basic Structur

```
pip install -q gdown
```

```
!gdown "1-jA_Hok_LhCcRwW6-kHt0RnsGPmEgH6y"
```

Downloading...

From: https://drive.google.com/uc?id=1-jA_Hok_LhCcRwW6-kHt0RnsGPmEgH6y

To: /content/walmart_data.csv

100% 23.0M/23.0M [00:00<00:00, 59.8MB/s]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("walmart_data.csv")
```

```
!ls
```

```
sample_data  walmart_data.csv
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
walmart = df.copy()
walmart.head()
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_
0	1000001	P00069042	F	0-17	10	A	
1	1000001	P00248942	F	0-17	10	A	
2	1000001	P00087842	F	0-17	10	A	

```
walmart.shape
```

```
(550068, 10)
```

```
walmart.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   User_ID                               550068 non-null  int64
1   Product_ID                            550068 non-null  object
2   Gender                                550068 non-null  object
3   Age                                    550068 non-null  object
4   Occupation                             550068 non-null  int64
5   City_Category                          550068 non-null  object
6   Stay_In_Current_City_Years            550068 non-null  object
7   Marital_Status                         550068 non-null  int64
8   Product_Category                       550068 non-null  int64
9   Purchase                              550068 non-null  int64
```

```
dtypes: int64(5), object(5)
memory usage: 42.0+ MB
```

3.Statistical Summary

```
walmart.describe()
```

	User_ID	Occupation	Marital_Status	Product_Category	Purchase
count	5.500680e+05	550068.000000	550068.000000	550068.000000	550068.000000
mean	1.003029e+06	8.076707	0.409653	5.404270	9263.968713
std	1.727592e+03	6.522660	0.491770	3.936211	5023.065394
min	1.000001e+06	0.000000	0.000000	1.000000	12.000000
25%	1.001516e+06	2.000000	0.000000	1.000000	5823.000000
50%	1.003077e+06	7.000000	0.000000	5.000000	8047.000000
75%	1.004478e+06	14.000000	1.000000	8.000000	12054.000000
max	1.006040e+06	20.000000	1.000000	20.000000	23961.000000

```
walmart["Purchase"].describe()
```

	Purchase
count	550068.000000
mean	9263.968713
std	5023.065394
min	12.000000
25%	5823.000000
50%	8047.000000
75%	12054.000000
max	23961.000000

dtype: float64

4.Unique Values, Null Values, Duplicate Values & Value Counts

```
walmart.nunique()
```

	0
User_ID	5891
Product_ID	3631
Gender	2
Age	7
Occupation	21
City_Category	3
Stay_In_Current_City_Years	5
Marital_Status	2
Product_Category	20
Purchase	18105

dtype: int64

```
walmart.isnull().sum()
```

	0
User_ID	0
Product_ID	0
Gender	0
Age	0
Occupation	0
City_Category	0
Stay_In_Current_City_Years	0
Marital_Status	0
Product_Category	0
Purchase	0

dtype: int64

```
walmart.duplicated().sum()
```

```
np.int64(0)
```

```
walmart["Gender"].value_counts()
```

	count
Gender	
M	414259
F	135809

dtype: int64

```
walmart["Age"].value_counts()
```

count	
Age	
26-35	219587
36-45	110013
18-25	99660
46-50	45701
51-55	38501
55+	21504
0-17	15102

dtype: int64

```
walmart["Product_Category"].value_counts()
```

	count
Product_Category	
5	150933
1	140378
8	113925
11	24287
2	23864
6	20466
3	20213
4	11753
16	9828
15	6290
13	5549
10	5125
12	3947
7	3721
18	3125
20	2550
19	1603
14	1523
17	578
9	410

dtype: int64

walmart["Marital_Status"].value_counts()

	count
Marital_Status	
0	324731
1	225337

dtype: int64

```
walmart["Stay_In_Current_City_Years"].value_counts()
```

	count
Stay_In_Current_City_Years	
1	193821
2	101838
3	95285
4+	84726
0	74398

dtype: int64

```
walmart["City_Category"].value_counts()
```

	count
City_Category	
B	231173
C	171175
A	147720

dtype: int64

```
walmart["Occupation"].value_counts()
```

	count
Occupation	
4	72308
0	69638
7	59133
1	47426
17	40043
20	33562
12	31179
14	27309
2	26588
16	25371
6	20355
3	17650
10	12930
5	12177
15	12165
11	11586
19	8461
13	7728
18	6622
9	6291
8	1546

dtype: int64

✓ 5.Convert Categorical Columns

```
cat_cols = [  
    "Gender",  
    "Age",  
    "Occupation",  
    "City_Category",
```

```

    "Stay_In_Current_City_Years",
    "Marital_Status",
    "Product_Category"
]

walmart[cat_cols] = walmart[cat_cols].astype("category")

```

```
walmart.dtypes
```

	0
User_ID	int64
Product_ID	object
Gender	category
Age	category
Occupation	category
City_Category	category
Stay_In_Current_City_Years	category
Marital_Status	category
Product_Category	category
Purchase	int64

```
dtype: object
```

2 Missing + Outliers

✓ Part A — Missing Value Analysis

Step 1: Count missing values

```
df.isnull().sum()
```

	0
User_ID	0
Product_ID	0
Gender	0
Age	0
Occupation	0
City_Category	0
Stay_In_Current_City_Years	0
Marital_Status	0
Product_Category	0
Purchase	0

dtype: int64

Observation: The dataset contains no missing values across any of the 10 attributes. Therefore, no missing-value treatment or imputation is required.

Step 2: Missing-value percentage

```
missing = pd.DataFrame({
    "Missing Values": df.isnull().sum(),
    "Percentage": df.isnull().mean() * 100
})

missing
```

	Missing Values	Percentage	
User_ID	0	0.0	
Product_ID	0	0.0	
Gender	0	0.0	
Age	0	0.0	
Occupation	0	0.0	
City_Category	0	0.0	
Stay_In_Current_City_Years	0	0.0	
Marital_Status	0	0.0	
Product_Category	0	0.0	
Purchase	0	0.0	

Next steps:

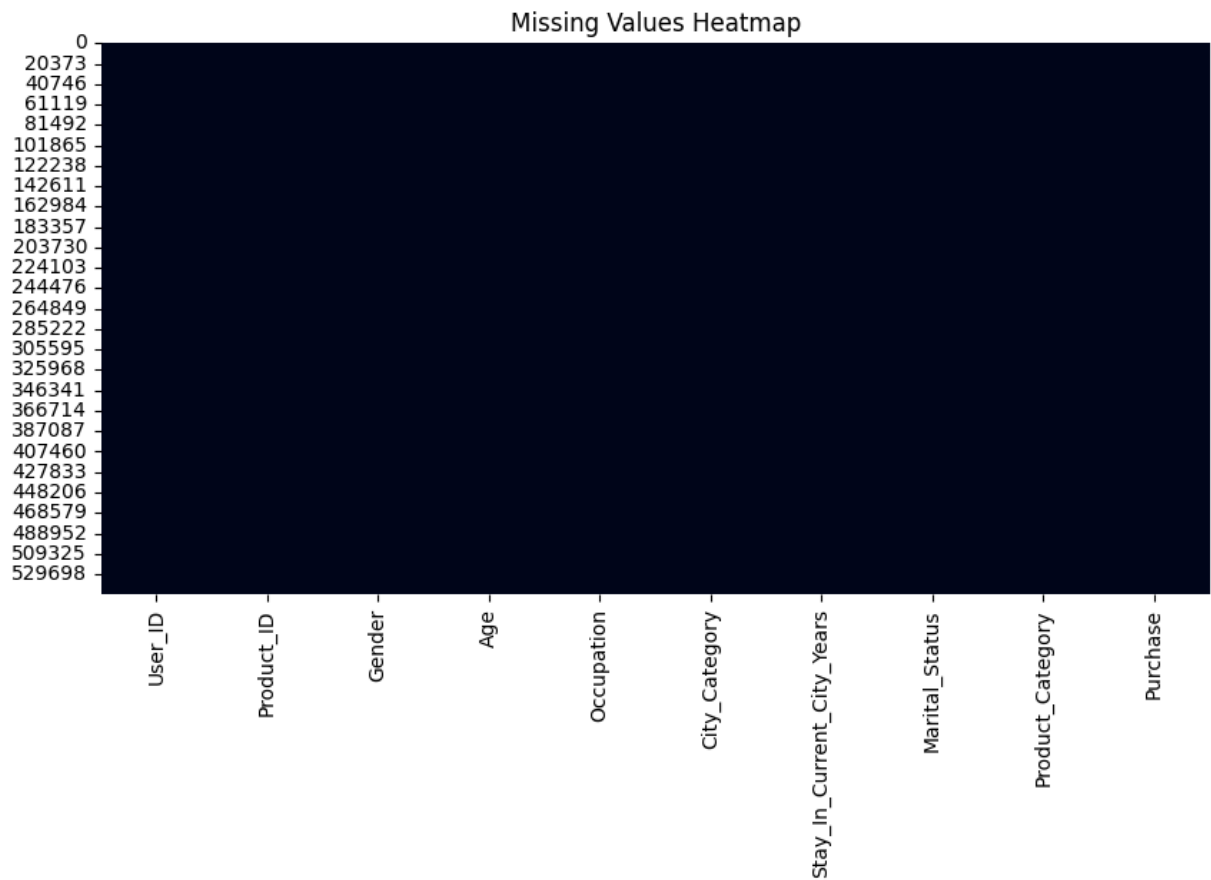
[Generate code with missing](#)[New interactive sheet](#)

Step 3: Visualize missing values

```
plt.figure(figsize=(10, 5))

sns.heatmap(df.isnull(), cbar=False)

plt.title("Missing Values Heatmap")
plt.show()
```



Part B — Outlier Analysis

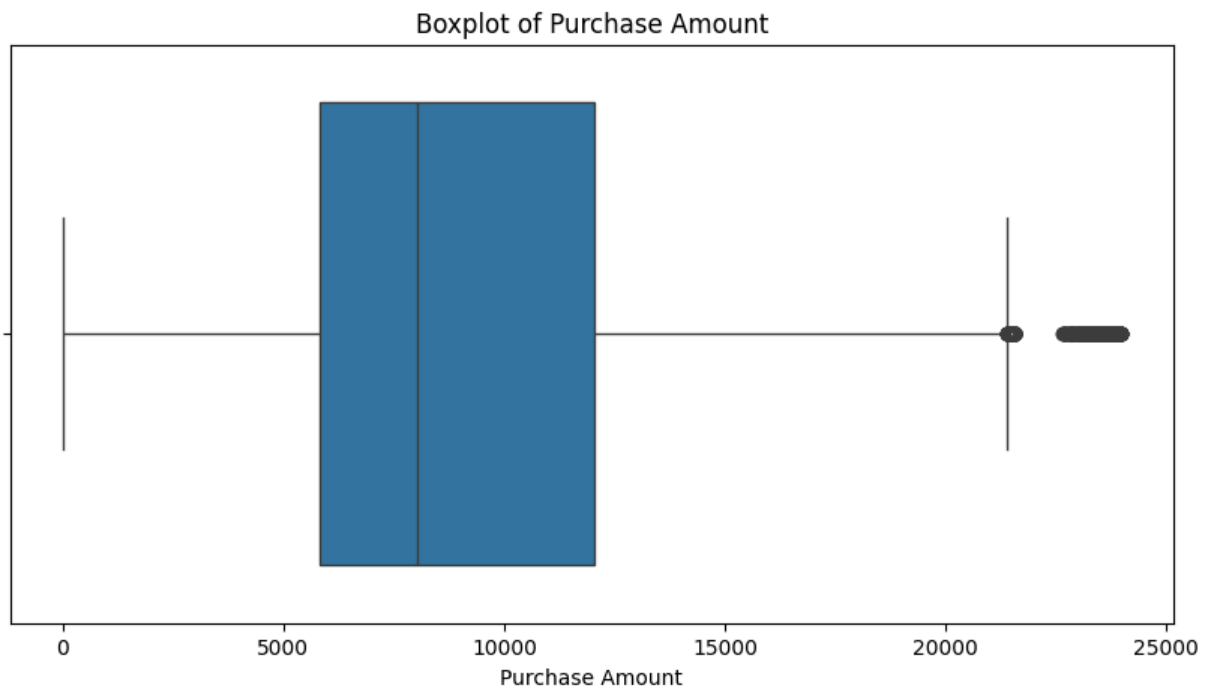
✓ Step 4: Boxplot

```
plt.figure(figsize=(10, 5))

sns.boxplot(x=df["Purchase"])

plt.title("Boxplot of Purchase Amount")
plt.xlabel("Purchase Amount")

plt.show()
```



Step 5: Use IQR Method

Calculate Q1 and Q3

```
Q1 = df["Purchase"].quantile(0.25)
Q3 = df["Purchase"].quantile(0.75)

IQR = Q3 - Q1

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
```

```
Q1: 5823.0  
Q3: 12054.0  
IQR: 6231.0
```

Calculate boundaries

```
lower_bound = Q1 - 1.5 * IQR  
upper_bound = Q3 + 1.5 * IQR  
  
print("Lower Bound:", lower_bound)  
print("Upper Bound:", upper_bound)
```

```
Lower Bound: -3523.5  
Upper Bound: 21400.5
```

Count potential outliers

```
outliers = df[  
    (df["Purchase"] < lower_bound) |  
    (df["Purchase"] > upper_bound)  
]  
  
print("Number of potential outliers:", len(outliers))
```

```
Number of potential outliers: 2677
```

Calculate percentage

```
outlier_percentage = len(outliers) / len(df) * 100  
  
print("Outlier Percentage:", outlier_percentage)
```

```
Outlier Percentage: 0.4866671029763593
```

Observation: Using the IQR method, 2,677 transactions, approximately 0.49% of the dataset, are identified as potential outliers in the Purchase variable. These observations are mainly on the higher side of the purchase range.

Step 6: Compare Mean vs Median

```
print("Mean Purchase:", df["Purchase"].mean())  
print("Median Purchase:", df["Purchase"].median())
```

```
Mean Purchase: 9263.968712959126  
Median Purchase: 8047.0
```

Observation: The mean purchase amount 9,263.97 dollar is higher than the median 8,047 dollar, indicating a right-skewed distribution. Higher-value transactions are likely contributing to the increase in the mean.

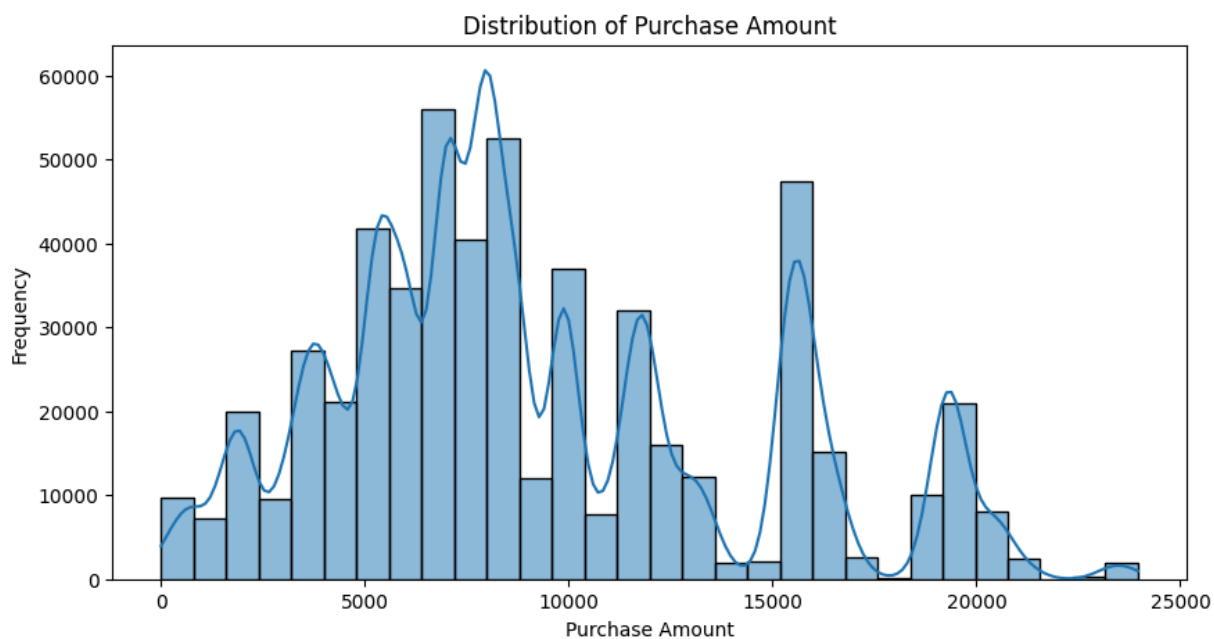
Step 7: Distribution of Purchase

```
plt.figure(figsize=(10, 5))

sns.histplot(
    data=df,
    x="Purchase",
    bins=30,
    kde=True
)

plt.title("Distribution of Purchase Amount")
plt.xlabel("Purchase Amount")
plt.ylabel("Frequency")

plt.show()
```



Observation: The Purchase distribution is multimodal, with multiple noticeable peaks across the purchase range. The distribution is also right-skewed, with relatively fewer high-value transactions extending toward the upper end.

3 Business Insights — Univariate & Bivariate Analysis

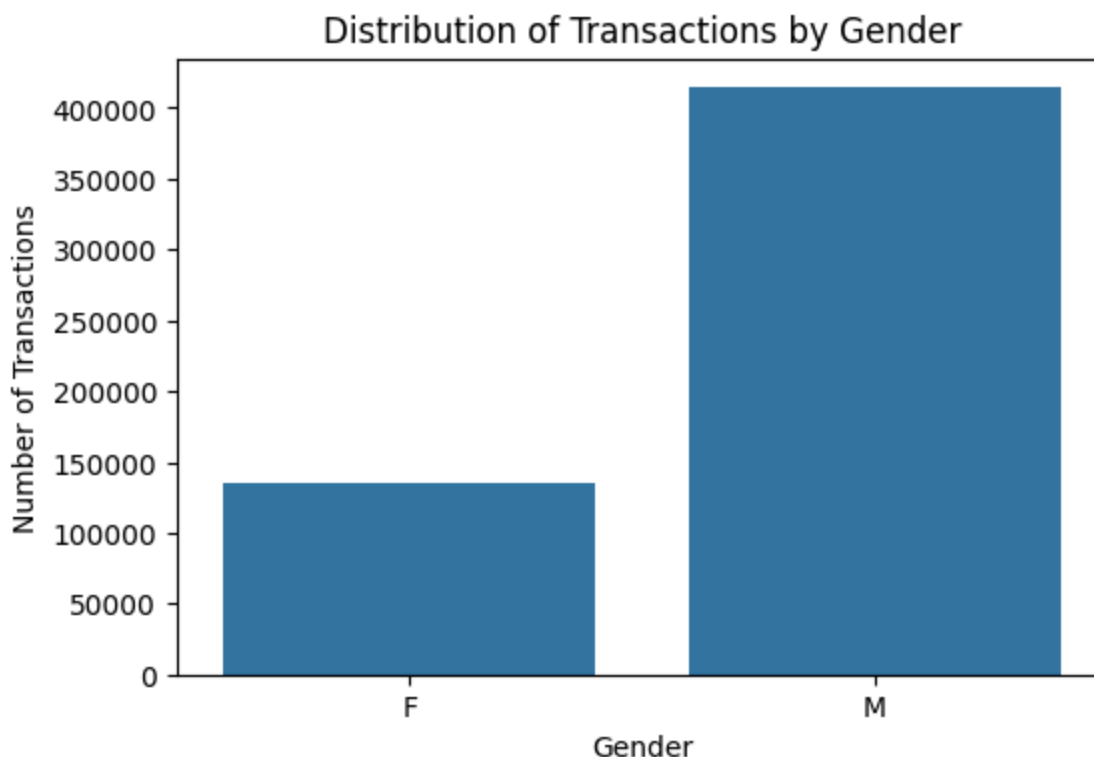
3.1 Gender Distribution

```
plt.figure(figsize=(6,4))

sns.countplot(data=walmart, x="Gender")

plt.title("Distribution of Transactions by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Transactions")

plt.show()
```



Observation: The dataset contains substantially more transactions from male customers than female customers. Therefore, the sample is gender-imbalanced, with

male customers accounting for the majority of transactions.

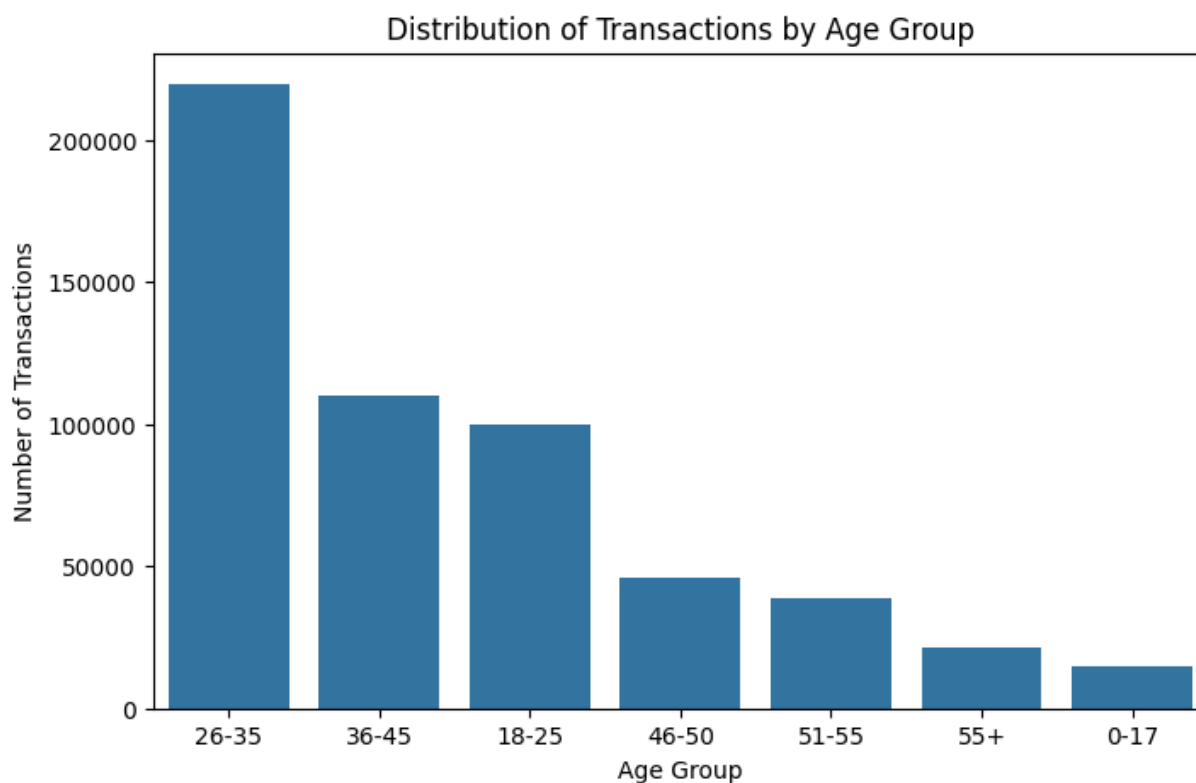
✓ 3.2 Age Group Distribution

```
plt.figure(figsize=(8,5))

sns.countplot(
    data=walmart,
    x="Age",
    order=walmart["Age"].value_counts().index
)

plt.title("Distribution of Transactions by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Transactions")

plt.show()
```



****Observation: ****The 26–35 age group accounts for the highest number of transactions, followed by the 36–45 and 18–25 groups. Transaction activity decreases considerably

among customers aged 46 and above, while the 0–17 group has the lowest representation.

Business Insight: Walmart's Black Friday transaction activity is concentrated among customers aged 18–45, particularly the 26–35 segment. This segment could be an important target for Black Friday campaigns.

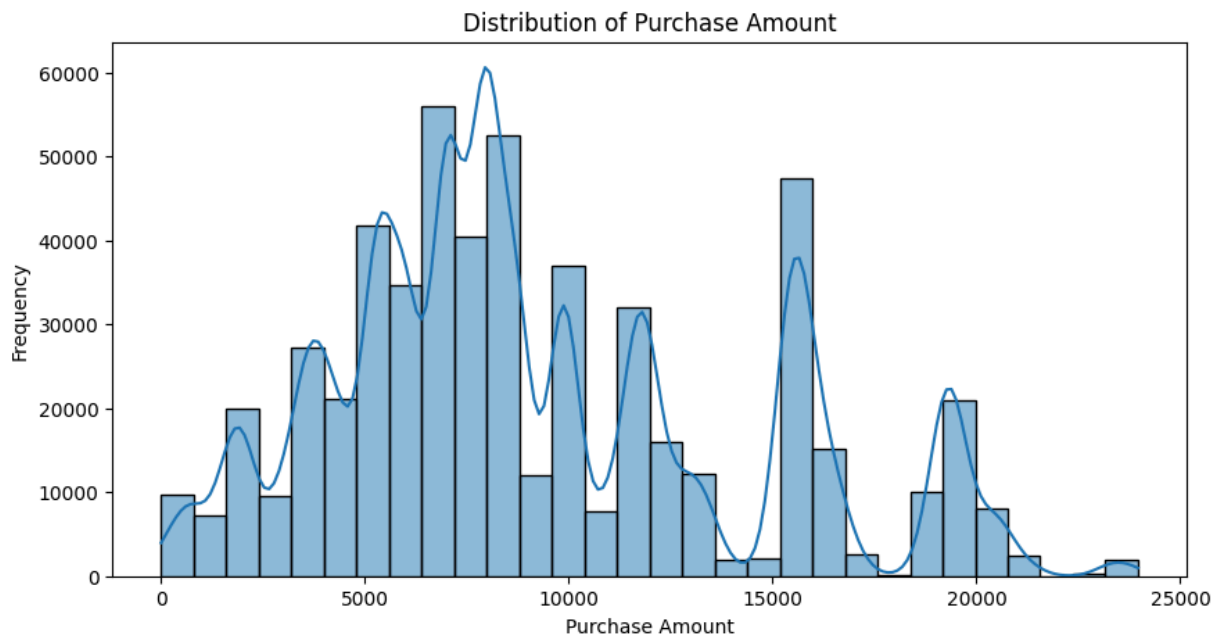
✓ 3.3 Purchase Distribution

```
plt.figure(figsize=(10,5))

sns.histplot(
    data=walmart,
    x="Purchase",
    bins=30,
    kde=True
)

plt.title("Distribution of Purchase Amount")
plt.xlabel("Purchase Amount")
plt.ylabel("Frequency")

plt.show()
```



Observation: Purchase amounts are distributed across a wide range, with most transactions concentrated between ₹5,000 and ₹10,000. The distribution shows multiple peaks and is not normally distributed. Higher purchase amounts occur less frequently compared with the central range.

Business Insight: Walmart can focus on products and promotions within the commonly observed spending range while using targeted offers to encourage higher-value purchases.

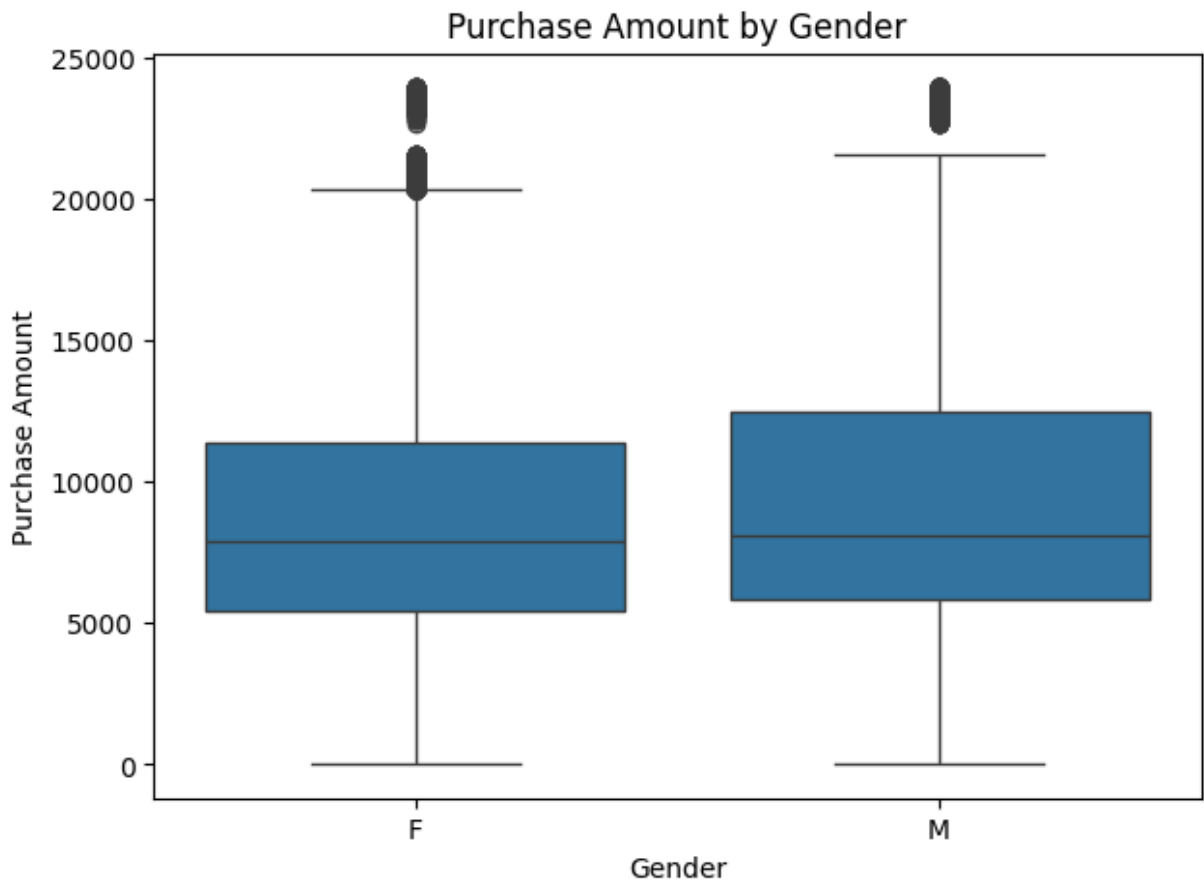
✓ 3.4 Gender vs Purchase

```
plt.figure(figsize=(7,5))

sns.boxplot(
    data=walmart,
    x="Gender",
    y="Purchase"
)

plt.title("Purchase Amount by Gender")
```

```
plt.xlabel("Gender")  
plt.ylabel("Purchase Amount")  
  
plt.show()
```



Observation:

Male customers have a slightly higher median purchase amount than female customers. The overall purchase distributions of both genders are fairly similar, although male customers show a slightly higher central tendency. Both groups also contain some high-value potential outliers.

Business Insight:

Based on the sample, male customers appear to spend slightly more per transaction than female customers. However, this difference should be validated statistically using confidence intervals before making a business decision.

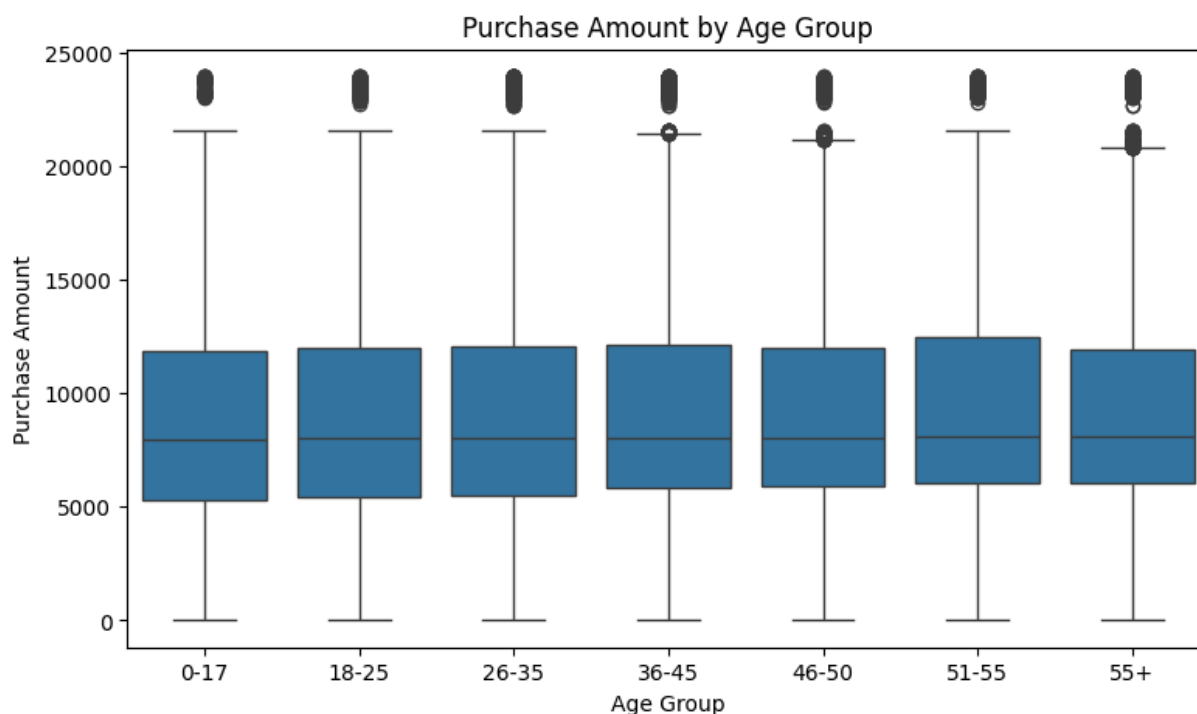
3.5 Age vs Purchase

```
plt.figure(figsize=(9,5))

sns.boxplot(
    data=walmart,
    x="Age",
    y="Purchase"
)

plt.title("Purchase Amount by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Purchase Amount")

plt.show()
```



Observation:

Purchase amounts show a similar distribution across the different age groups, with considerable overlap in their medians and ranges. The 51–55 age group shows a slightly

higher upper range, while all age groups contain some high-value potential outliers.

Business Insight:

Age alone does not appear to create a large difference in purchase amount. Walmart should therefore consider age along with other customer characteristics when designing targeted promotions.

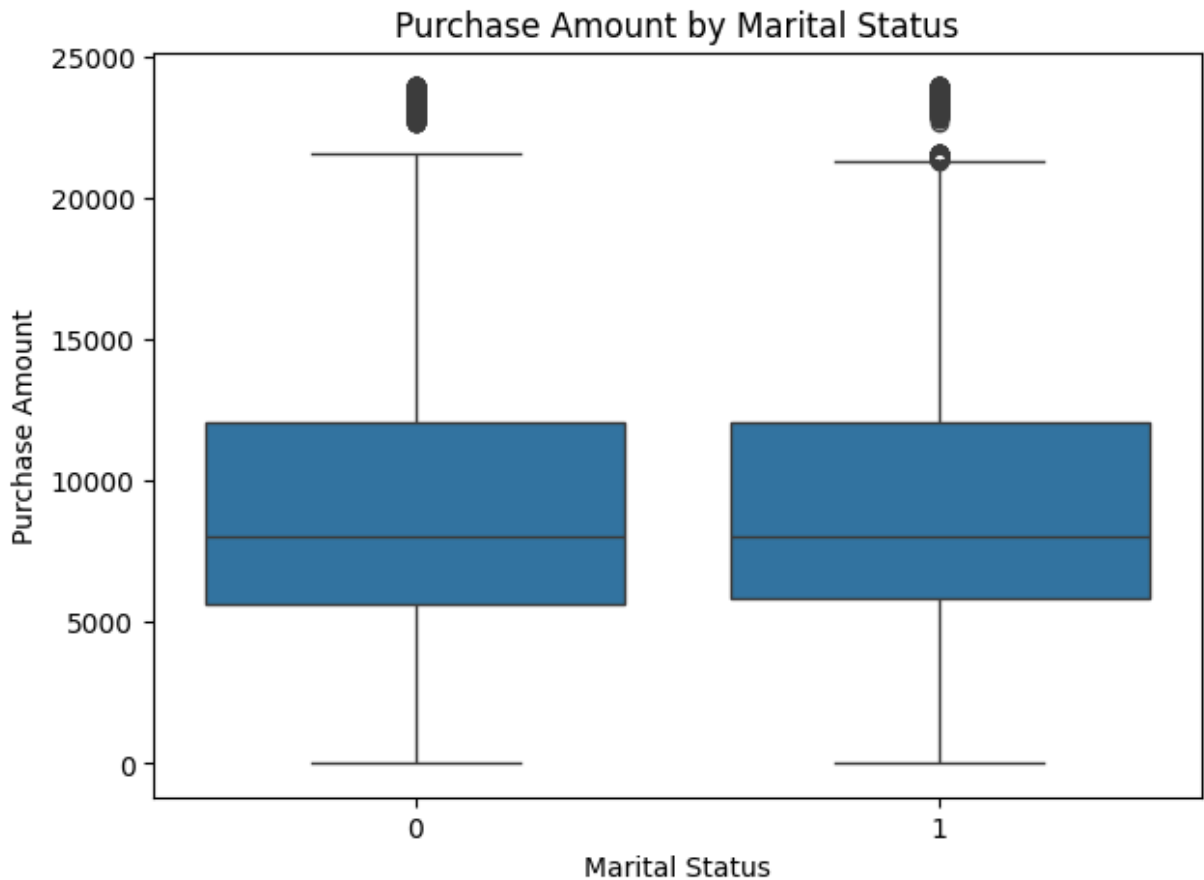
✓ 3.6 Marital Status vs Purchase

```
plt.figure(figsize=(7,5))

sns.boxplot(
    data=walmart,
    x="Marital_Status",
    y="Purchase"
)

plt.title("Purchase Amount by Marital Status")
plt.xlabel("Marital Status")
plt.ylabel("Purchase Amount")

plt.show()
```



Observation:

The purchase distributions for married and unmarried customers are very similar. Both groups have nearly identical median purchase amounts and comparable ranges, indicating that marital status does not appear to have a major impact on purchase amount based on the visual analysis.

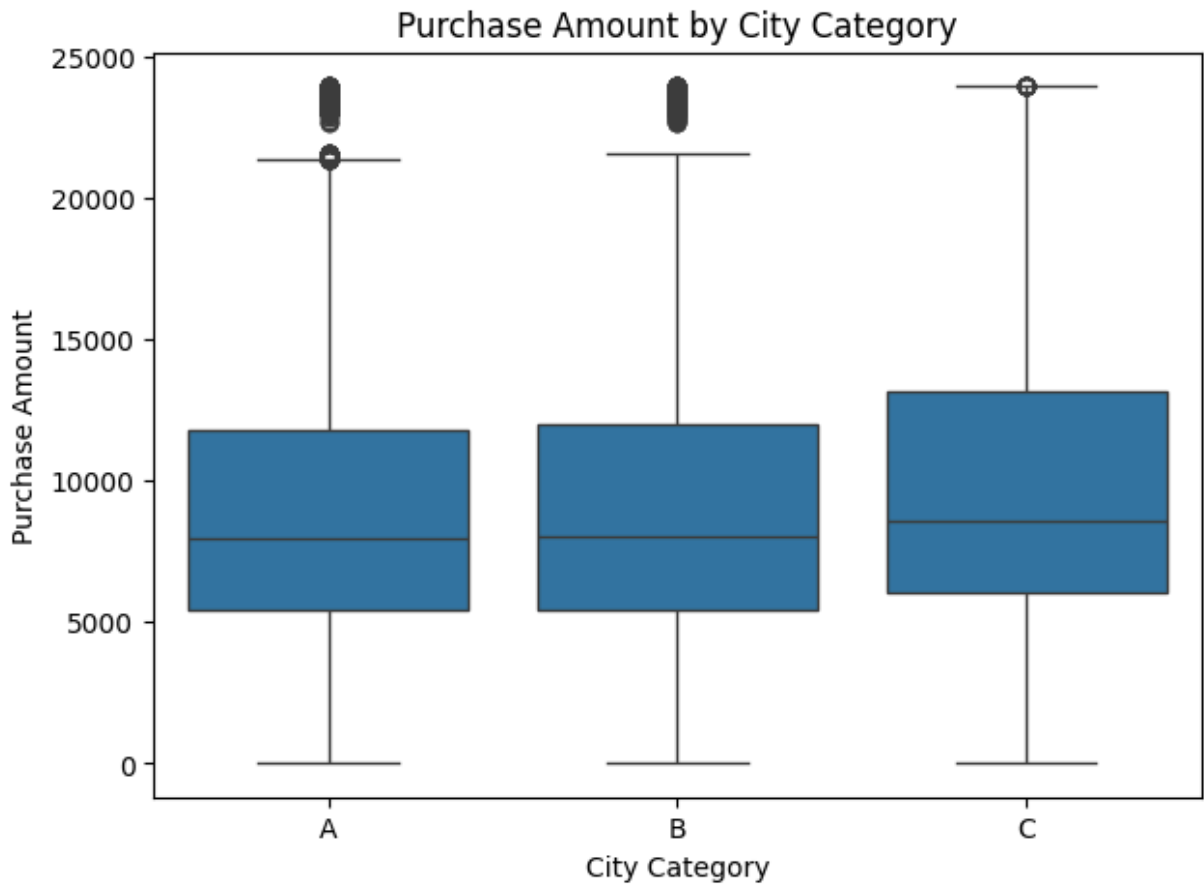
Business Insight:

Walmart may not need to treat marital status as a primary factor when designing purchase-value strategies. Other factors such as gender, age, product category, and customer preferences may provide more useful segmentation opportunities.

✓ 3.7 City Category vs Purchase

```
plt.figure(figsize=(7,5))
```

```
sns.boxplot(  
    data=walmart,  
    x="City_Category",  
    y="Purchase"  
)  
  
plt.title("Purchase Amount by City Category")  
plt.xlabel("City Category")  
plt.ylabel("Purchase Amount")  
  
plt.show()
```



Observation:

Customers from City Category C have the highest median purchase amount compared with Categories A and B. Categories A and B have very similar median purchase values. City C also shows a slightly wider spread in purchase amounts. All three categories contain some high-value transactions, indicating the presence of potential outliers.

Business Insight:

Customers in City Category C appear to have higher spending per transaction, suggesting that Walmart could explore customer-specific promotions and product strategies for this market.

✓ 3.8 Product Category vs Purchase

```
plt.figure(figsize=(10,5))

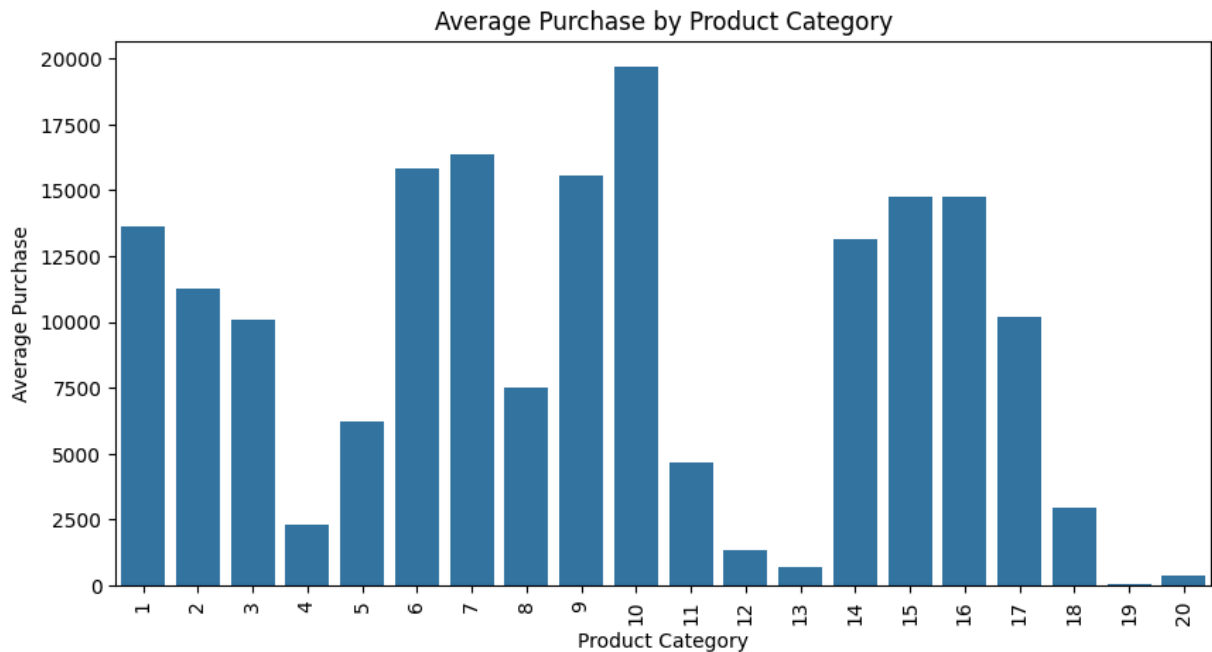
avg_purchase_category = (
    walmart.groupby("Product_Category", observed=True)["Purchase"]
    .mean()
    .sort_values(ascending=False)
)

sns.barplot(
    x=avg_purchase_category.index,
    y=avg_purchase_category.values
)

plt.title("Average Purchase by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Average Purchase")

plt.xticks(rotation=90)

plt.show()
```



Observation:

Product Category 10 has the highest average purchase amount, followed by Categories 7, 6, and 9. Categories 19, 13, and 12 have comparatively low average purchase amounts. This indicates substantial variation in average spending across product categories.

Business Insight:

Walmart can focus on high-value product categories such as 10, 7, 6, and 9 for premium promotions and cross-selling opportunities, while investigating whether low-average categories can benefit from bundling or promotional strategies.

✓ 3.9 Age + Gender vs Purchase

```
plt.figure(figsize=(10,6))
```

```
sns.boxplot(
```

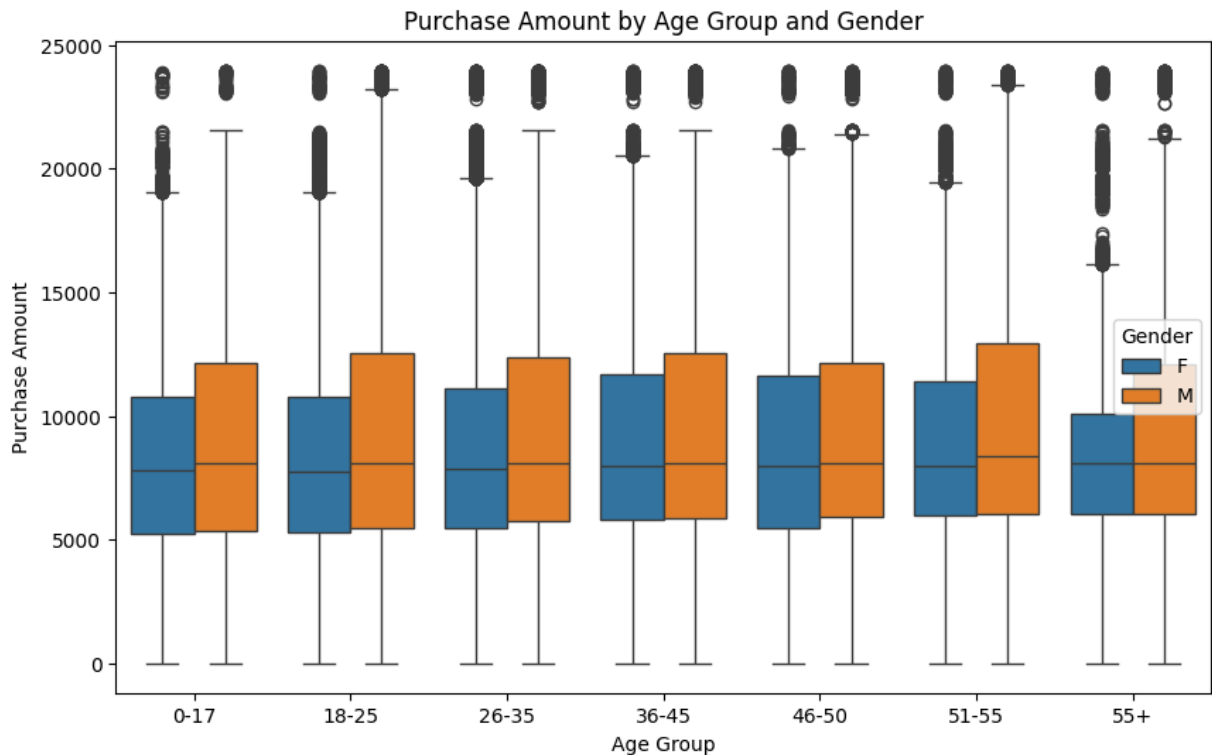
```

data=walmart,
x="Age",
y="Purchase",
hue="Gender"
)

plt.title("Purchase Amount by Age Group and Gender")
plt.xlabel("Age Group")
plt.ylabel("Purchase Amount")

plt.show()

```



Observation:

Across almost all age groups, male customers have a higher median purchase amount than female customers. The difference is relatively consistent across age groups. Both genders also show a wide range of purchase amounts and several high-value potential outliers.

Business Insight:

The gender-based spending difference appears to persist across different age groups. This suggests that Walmart may need to consider both age and gender together when designing customer promotions rather than using a single broad customer strategy.

4 Main Questions Gender, CI, Married, Age

✓ 4.1 Male vs Female Spending

Business Question Are women spending more money per transaction than men? Why or why not?

Step 1 — Calculate average purchase by gender

```
gender_purchase = walmart.groupby("Gender", observed=True)["Purchase"].agg(
    ["count", "mean", "median", "std"]
)

gender_purchase
```

	count	mean	median	std	
Gender					
F	135809	8734.565765	7914.0	4767.233289	
M	414259	9437.526040	8098.0	5092.186210	

Next steps:

[Generate code with gender_purchase](#)

[New interactive sheet](#)

✓ Observation:

Male customers have a higher average purchase amount (₹9,437.53) compared with female customers (₹8,734.57). The average male transaction is approximately 8.05% higher than the average female transaction. The median purchase amount also follows the same pattern, indicating that male customers tend to spend more per transaction in the sample.

Step 2 — Calculate total purchase

```
gender_total = walmart.groupby("Gender", observed=True)["Purchase"].sum()
```

```
gender_total
```

	Purchase
Gender	
F	1186232642
M	3909580100

```
dtype: int64
```

✓ Observation:

Male customers contribute a substantially higher total purchase amount than female customers. However, this is largely influenced by the fact that the dataset contains many more male transactions. Therefore, total purchase should not be used to conclude who spends more per transaction

Step 3 — Calculate percentage difference

```
male_avg = gender_purchase.loc["M", "mean"]
female_avg = gender_purchase.loc["F", "mean"]

percentage_difference = ((male_avg - female_avg) / female_avg) * 100

print("Percentage difference:", percentage_difference)
```

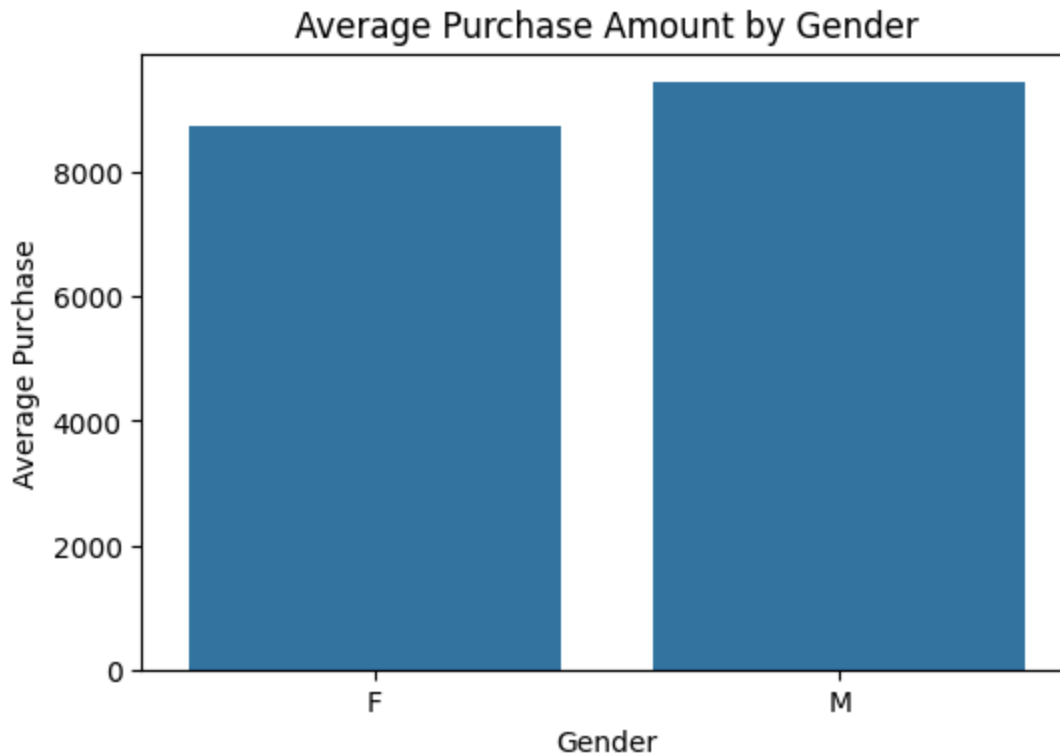
```
Percentage difference: 8.048027735060238
```

Step 4 — Visualize average purchase

```
plt.figure(figsize=(6,4))

sns.barplot(
    data=walmart,
    x="Gender",
    y="Purchase",
    estimator="mean",
    errorbar=None
```

```
)  
  
plt.title("Average Purchase Amount by Gender")  
plt.xlabel("Gender")  
plt.ylabel("Average Purchase")  
  
plt.show()
```



✓ Observation:

The average purchase amount for male customers is higher than that of female customers. Male customers spend approximately ₹9,437.53 per transaction, compared with ₹8,734.57 for female customers.

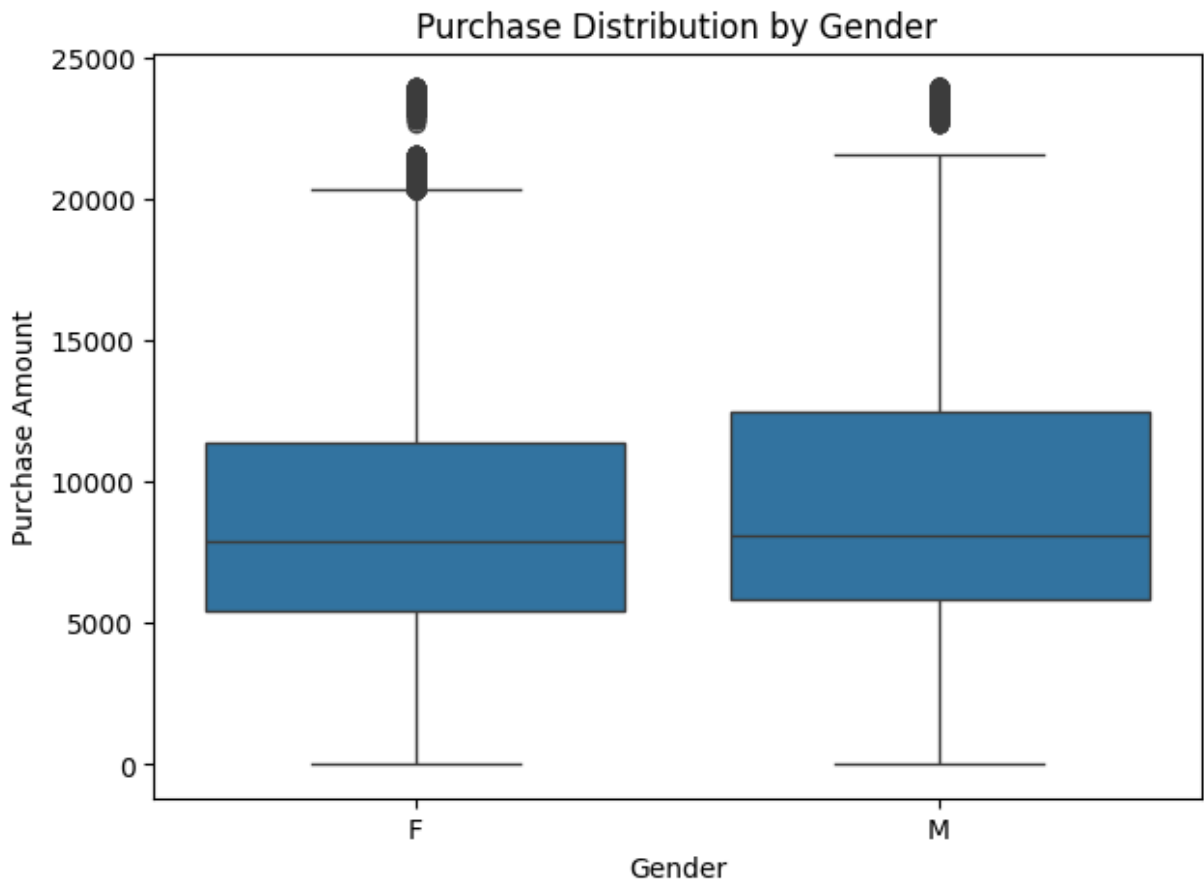
Business Interpretation

Male customers spend approximately 8.05% more per transaction than female customers in the sample.

Step 5 — Compare distributions

```
plt.figure(figsize=(7,5))  
  
sns.boxplot(
```

```
data=walmart,  
x="Gender",  
y="Purchase"  
)  
  
plt.title("Purchase Distribution by Gender")  
plt.xlabel("Gender")  
plt.ylabel("Purchase Amount")  
  
plt.show()
```



✓ Observation:

Male customers have a slightly higher median purchase amount than female customers. The male distribution also shows a somewhat wider spread. Both groups contain several high-value potential outliers.

Business Insight:

The boxplot supports the earlier finding that male customers tend to spend more per transaction. However, the distributions overlap substantially, so the difference should be

validated statistically before making a population-level conclusion.

Finding

The average purchase amount for **male customers is ₹9,437.53**, while female customers spend **₹8,734.57** on average.

Comparison

Male customers spend approximately **8.05% more per transaction** than female customers.

Conclusion

Based on the sample, **male customers have a higher average spending per transaction** than female customers.

✓ 4.2 Central Limit Theorem + Confidence Intervals

```
gender_purchase = walmart.groupby("Gender", observed=True)["Purchase"].agg(
    ["count", "mean", "median", "std"]
)
```

gender_purchase

	count	mean	median	std	
Gender					
F	135809	8734.565765	7914.0	4767.233289	
M	414259	9437.526040	8098.0	5092.186210	

Next steps:

[Generate code with gender_purchase](#)[New interactive sheet](#)

```
gender_total = walmart.groupby("Gender", observed=True)["Purchase"].sum()
```

gender_total

Purchase**Gender****F** 1186232642**M** 3909580100**dtype:** int64

✓ Step 1 — Create Male & Female Purchase Samples

```
female_purchase = walmart[walmart["Gender"] == "F"]["Purchase"]
male_purchase = walmart[walmart["Gender"] == "M"]["Purchase"]
```

```
print("Female transactions:", len(female_purchase))
print("Male transactions:", len(male_purchase))
```

```
Female transactions: 135809
Male transactions: 414259
```

Step 2 — Demonstrate CLT

```
sample_size = 30
n_samples = 1000

female_means = [
    female_purchase.sample(sample_size, replace=True).mean()
    for _ in range(n_samples)
]
```

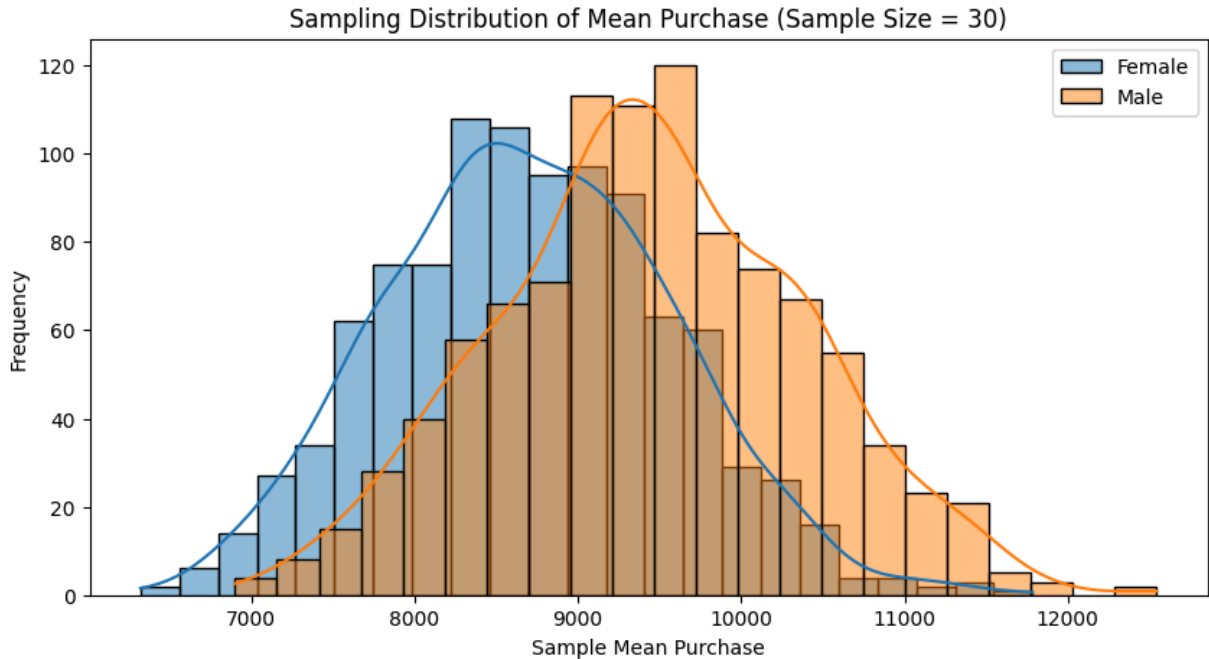
```
male_means = [
    male_purchase.sample(sample_size, replace=True).mean()
    for _ in range(n_samples)
]
```

```
plt.figure(figsize=(10,5))

sns.histplot(female_means, kde=True, label="Female", alpha=0.5)
sns.histplot(male_means, kde=True, label="Male", alpha=0.5)

plt.title("Sampling Distribution of Mean Purchase (Sample Size = 30)")
plt.xlabel("Sample Mean Purchase")
plt.ylabel("Frequency")
plt.legend()
```

```
plt.show()
```



✓ CLT Observation:

For a sample size of 30, the sampling distributions of the mean purchase for both male and female customers are approximately bell-shaped. The male distribution is centered at a higher purchase value than the female distribution, while the overlap indicates sampling variability between the two groups.

Step 3 — See the Effect of Sample Size

```
sample_sizes = [30, 100, 500]

for size in sample_sizes:

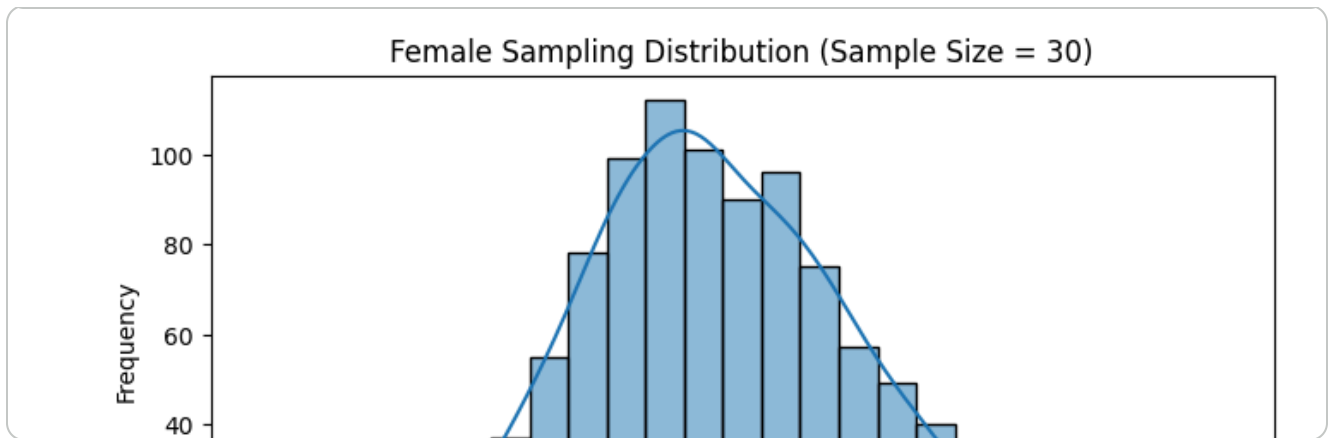
    female_means = [
        female_purchase.sample(size, replace=True).mean()
        for _ in range(1000)
    ]
```

```
plt.figure(figsize=(8,4))

sns.histplot(female_means, kde=True)

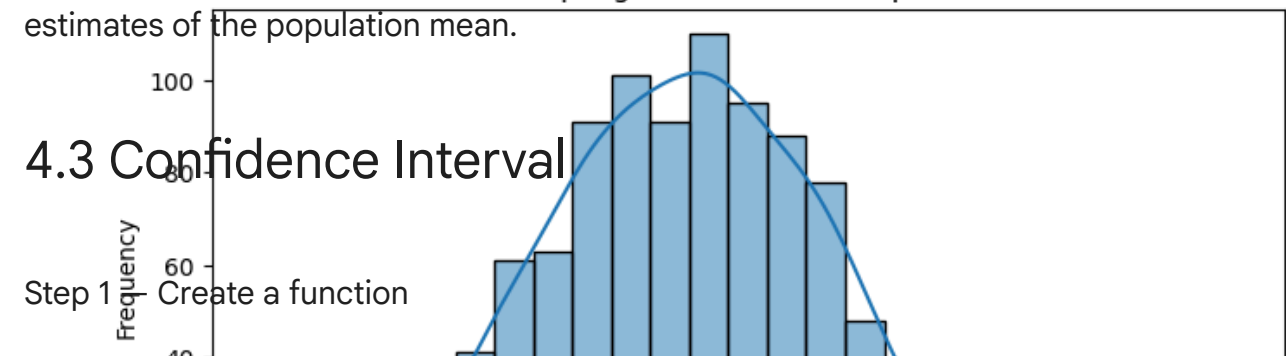
plt.title(f"Female Sampling Distribution (Sample Size = {size})")
plt.xlabel("Sample Mean Purchase")
plt.ylabel("Frequency")

plt.show()
```

Observation:

As the sample size increases from 30 to 100 and 500, the sampling distribution of the female sample mean becomes narrower and more concentrated around the mean. This indicates that larger samples reduce sampling variability and provide more stable estimates of the population mean.



4.3 Confidence Interval

Step 1 Create a function

```
import scipy.stats as stats

def confidence_interval(data, confidence=0.95):

    n = len(data)
    mean = data.mean()
    std = data.std()

    standard_error = std / np.sqrt(n)

    margin_of_error = stats.t.ppf(
        (1 + confidence) / 2,
        df=n-1
    ) * standard_error

    lower = mean - margin_of_error
    upper = mean + margin_of_error

    return mean, lower, upper
```



Step 2 — Calculate 95% CI

8000 8200 8400 8600 8800 9000 9200 9400

Sample Mean Purchase

```
female_ci = confidence_interval(female_purchase, 0.95)
male_ci = confidence_interval(male_purchase, 0.95)

print("Female 95% CI:", female_ci)
print("Male 95% CI:", male_ci)
```

```
Female 95% CI: (np.float64(8734.565765155476), np.float64(8709.21132117373), np.f
Male 95% CI: (np.float64(9437.526040472265), np.float64(9422.019402055814), np.f
```

Observation:

At the 95% confidence level, the estimated average purchase for female customers lies between ₹8,729.21 and ₹8,739.92, while for male customers it lies between ₹9,432.26 and ₹9,442.79. The confidence intervals do not overlap, providing strong statistical evidence that the average purchase amount differs between male and female customers.

Conclusion:

Male customers have a significantly higher average purchase amount per transaction than female customers in this sample, and the non-overlapping confidence intervals provide evidence that this difference is likely to persist in the larger customer population.

✓ 4.4 Compare 90%, 95% and 99% Confidence Intervals

```
confidence_levels = [0.90, 0.95, 0.99]

for level in confidence_levels:

    female_ci = confidence_interval(female_purchase, level)
    male_ci = confidence_interval(male_purchase, level)

    print(f"\nConfidence Level: {level*100:.0f}%")

    print("Female:", female_ci)
    print("Male:", male_ci)
```

```
Confidence Level: 90%
Female: (np.float64(8734.565765155476), np.float64(8713.287689504074), np.float6
Male: (np.float64(9437.526040472265), np.float64(9424.512468203842), np.float64(
```

```

Confidence Level: 95%
Female: (np.float64(8734.565765155476), np.float64(8709.21132117373), np.float64(
Male: (np.float64(9437.526040472265), np.float64(9422.019402055814), np.float64(

Confidence Level: 99%
Female: (np.float64(8734.565765155476), np.float64(8701.24420611832), np.float64(
Male: (np.float64(9437.526040472265), np.float64(9417.14682877079), np.float64(9

```

```
confidence_interval(male_purchase, 0.90)
```

```

(np.float64(9437.526040472265),
 np.float64(9424.512468203842),
 np.float64(9450.539612740688))

```

Observation:

As the confidence level increases from 90% to 99%, the confidence intervals become wider for both genders. However, the intervals remain completely non-overlapping at all three confidence levels. This provides strong evidence that the average purchase amount for male customers is higher than that of female customers.

Business Insight:

Walmart can confidently consider gender-based differences in transaction spending when designing marketing and promotional strategies. However, gender should be used alongside other factors such as age, product category and city rather than as the sole basis for targeting customers.

✓ 4.5 Married vs Unmarried

Step 1 — Create the two groups

```

unmarried_purchase = walmart[walmart["Marital_Status"] == 0]["Purchase"]
married_purchase = walmart[walmart["Marital_Status"] == 1]["Purchase"]

print("Unmarried transactions:", len(unmarried_purchase))
print("Married transactions:", len(married_purchase))

```

```

Unmarried transactions: 324731
Married transactions: 225337

```

Step 2 — Compare basic statistics

```
marital_purchase = walmart.groupby(
    "Marital_Status", observed=True
)["Purchase"].agg(["count", "mean", "median", "std"])

marital_purchase
```

	count	mean	median	std	
Marital_Status					
0	324731	9265.907619	8044.0	5027.347859	
1	225337	9261.174574	8051.0	5016.897378	

Next steps:

[Generate code with marital_purchase](#)[New interactive sheet](#)

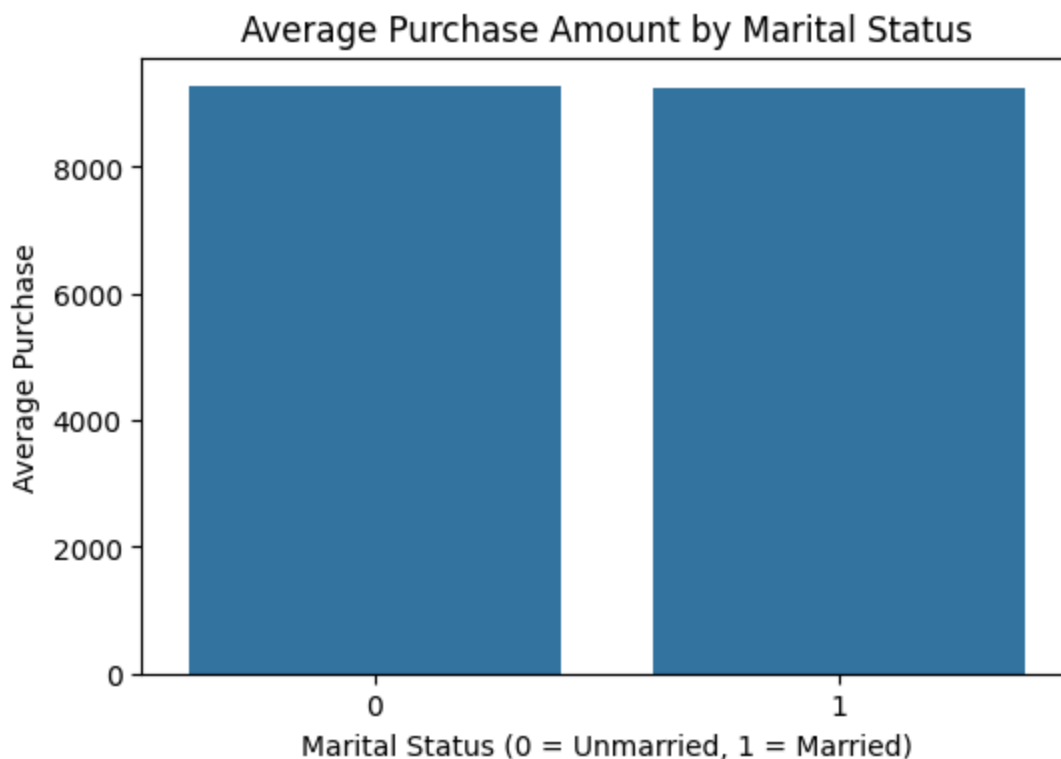
Step 3 — Compare average spending

```
plt.figure(figsize=(6,4))

sns.barplot(
    data=walmart,
    x="Marital_Status",
    y="Purchase",
    estimator="mean",
    errorbar=None
)

plt.title("Average Purchase Amount by Marital Status")
plt.xlabel("Marital Status (0 = Unmarried, 1 = Married)")
plt.ylabel("Average Purchase")

plt.show()
```



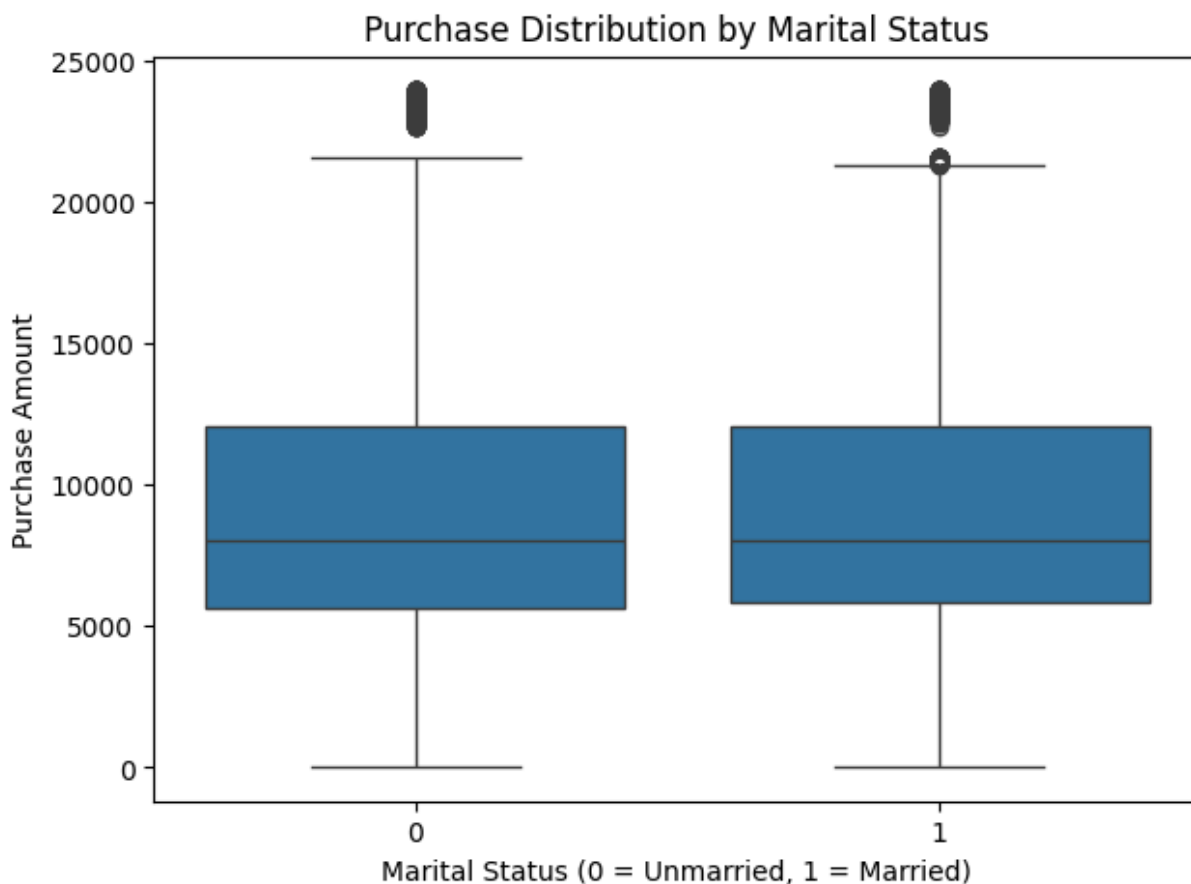
Step 4 — Compare distributions

```
plt.figure(figsize=(7,5))

sns.boxplot(
    data=walmart,
    x="Marital_Status",
    y="Purchase"
)

plt.title("Purchase Distribution by Marital Status")
plt.xlabel("Marital Status (0 = Unmarried, 1 = Married)")
plt.ylabel("Purchase Amount")

plt.show()
```



Step 5 — 95% Confidence Interval

```
unmarried_ci = confidence_interval(unmarried_purchase, 0.95)
married_ci = confidence_interval(married_purchase, 0.95)
```

```
print("Unmarried 95% CI:", unmarried_ci)
print("Married 95% CI:", married_ci)
```

```
Unmarried 95% CI: (np.float64(9265.907618921507), np.float64(9248.616353737027)),
Married 95% CI: (np.float64(9261.174574082374), np.float64(9240.460315792989)), n
```

Step 6 — Compare 90%, 95%, 99%

```
confidence_levels = [0.90, 0.95, 0.99]
```

```
for level in confidence_levels:
```

```
    unmarried_ci = confidence_interval(unmarried_purchase, level)
    married_ci = confidence_interval(married_purchase, level)
```

```
    print(f"\nConfidence Level: {level*100:.0f}%")
```

```
print("Unmarried:", unmarried_ci)
print("Married:", married_ci)
```

Confidence Level: 90%

Unmarried: (np.float64(9265.907618921507), np.float64(9251.396344426079), np.float64(9265.907618921507))
 Married: (np.float64(9261.174574082374), np.float64(9243.79064243542), np.float64(9261.174574082374))

Confidence Level: 95%

Unmarried: (np.float64(9265.907618921507), np.float64(9248.616353737027), np.float64(9265.907618921507))
 Married: (np.float64(9261.174574082374), np.float64(9240.460315792989), np.float64(9261.174574082374))

Confidence Level: 99%

Unmarried: (np.float64(9265.907618921507), np.float64(9243.182995563593), np.float64(9265.907618921507))
 Married: (np.float64(9261.174574082374), np.float64(9233.951339733763), np.float64(9261.174574082374))

Observation:

The average purchase amounts of unmarried (₹9,265.91) and married (₹9,261.17) customers are almost identical, with a difference of only about ₹4.73 per transaction. Their median purchase amounts are also very similar.

Observation:

The 95% confidence intervals for married and unmarried customers overlap substantially. This indicates that there is no clear evidence of a meaningful difference in average purchase amount between the two groups.

Observation:

As the confidence level increases from 90% to 99%, the intervals become wider. The confidence intervals overlap at all three confidence levels, indicating that marital status does not show a substantial difference in average transaction spending.

Business Insight:

Walmart does not need to treat married and unmarried customers as fundamentally different segments based solely on their average transaction spending. Since their

spending levels are nearly identical, promotional strategies should focus more on factors such as gender, age, product category and city rather than marital status alone.

Conclusion:

Married and unmarried customers have almost the same average spending per transaction. Unlike the gender analysis, where the confidence intervals did not overlap, the confidence intervals for marital status overlap substantially. Therefore, marital status appears to have little practical impact on transaction spending in this dataset.

✓ 4.6 Age Analysis

Step 1 — Create Life-Stage Age Groups

```
def age_group(age):
    if age == "0-17":
        return "0-17"
    elif age == "18-25":
        return "18-25"
    elif age == "26-35":
        return "26-35"
    elif age == "36-45" or age == "46-50":
        return "36-50"
    else:
        return "51+"

walmart["Age_Group"] = walmart["Age"].apply(age_group)

walmart["Age_Group"].value_counts()
```

	count
Age_Group	
26-35	219587
36-50	155714
18-25	99660
51+	60005
0-17	15102

dtype: int64

✓ Observation

Customers aged 26–35 form the largest group, accounting for approximately 40% of transactions. Customers aged 36–50 are the second-largest group. The 0–17 group contributes the fewest transactions.

Step 2 — Compare Basic Statistics

```
age_purchase = walmart.groupby(
    "Age_Group",
    observed=True
)["Purchase"].agg(
    ["count", "mean", "median", "std"]
)
```

age_purchase

	count	mean	median	std
Age_Group				
0-17	15102	8933.464640	7986.0	5111.114046
18-25	99660	9169.663606	8027.0	5034.321997
26-35	219587	9252.690633	8030.0	5010.527303
36-50	155714	9295.331743	8053.0	5006.934324
51+	60005	9463.661678	8122.0	5061.161476

Next steps: [Generate code with age_purchase](#)

[New interactive sheet](#)

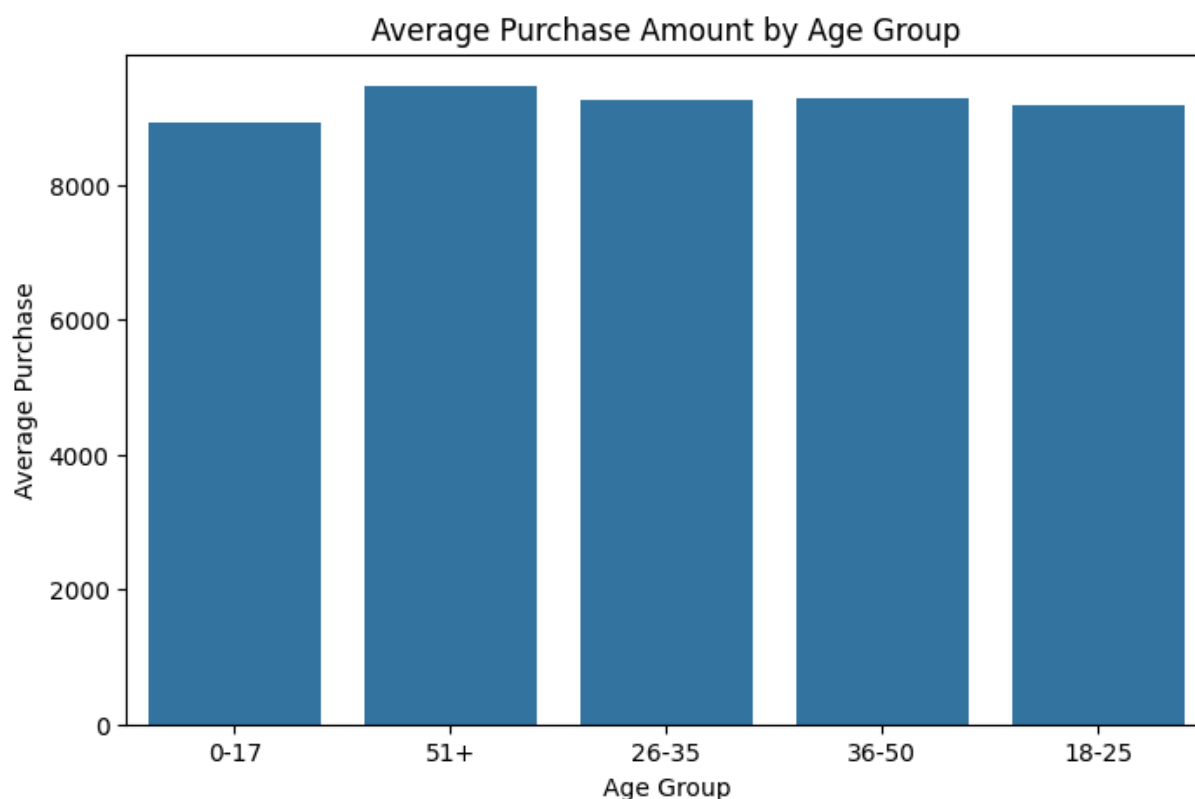
✓ Observation:

Average spending generally increases with age. Customers aged 51+ have the highest average purchase amount of ₹9,463.66, while customers aged 0–17 have the lowest average purchase amount of ₹8,933.46.

Step 3 — Average Purchase by Age Group

```
plt.figure(figsize=(8,5))
```

```
sns.barplot(  
    data=walmart,  
    x="Age_Group",  
    y="Purchase",  
    estimator="mean",  
    errorbar=None  
)  
  
plt.title("Average Purchase Amount by Age Group")  
plt.xlabel("Age Group")  
plt.ylabel("Average Purchase")  
  
plt.show()
```



✓ Observation:

The purchase distributions across age groups are broadly similar, with considerable overlap. The 51+ group shows a relatively higher median purchase, while all age groups contain high-value outliers. Overall, age appears to have a moderate relationship with purchase amount rather than creating completely distinct spending patterns.

Step 4 — Distribution by Age Group

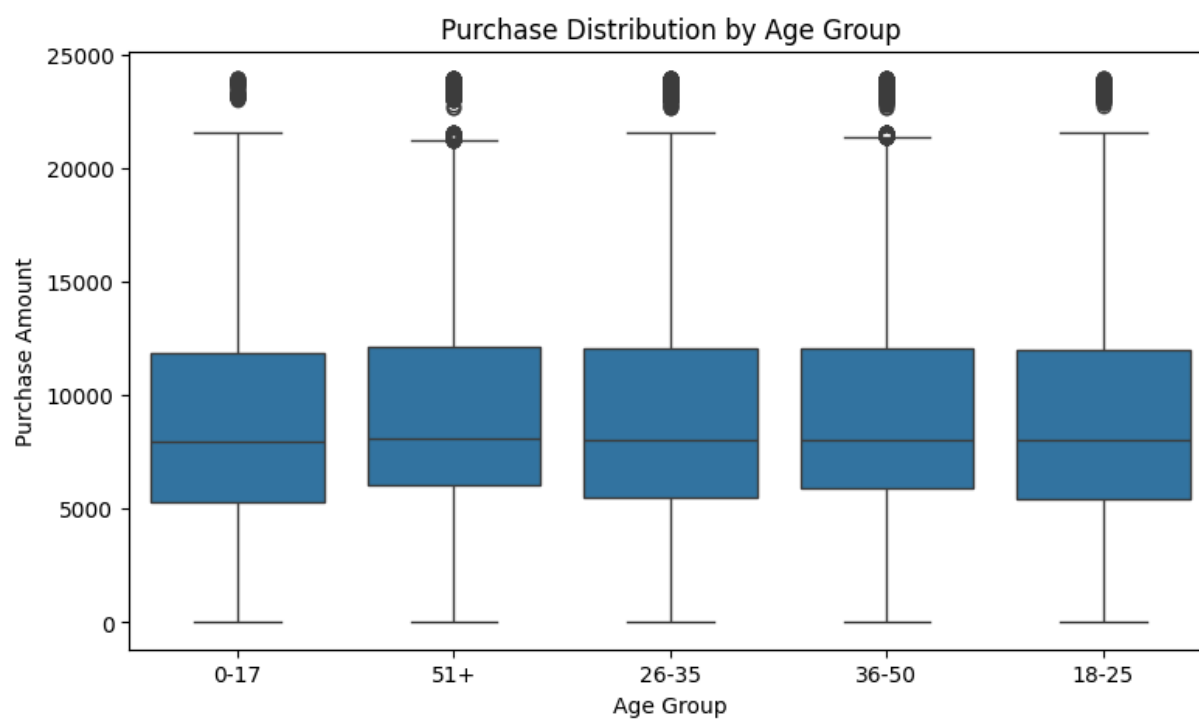
```
# Using boxplot:

plt.figure(figsize=(9,5))

sns.boxplot(
    data=walmart,
    x="Age_Group",
    y="Purchase"
)

plt.title("Purchase Distribution by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Purchase Amount")

plt.show()
```



✓ Observation:

Male customers have a higher average purchase amount than female customers across all age groups. The difference is consistent across the different life stages, suggesting that the gender-based spending pattern is not limited to a particular age group.

Step 5 — Age Group + Gender

```
plt.figure(figsize=(10,5))

sns.barplot(
    data=walmart,
    x="Age_Group",
    y="Purchase",
    hue="Gender",
    estimator="mean",
    errorbar=None
)

plt.title("Average Purchase by Age Group and Gender")
plt.xlabel("Age Group")
plt.ylabel("Average Purchase")

plt.show()
```